

ManaMind Case Study - Navlika Singh

Please find attached link to the [code](#)

Task 1: Skill Definition

Every skill in the library must conform to the following schema. The schema is stored as a structured JSON object.

Please take note of the following points:

1. The 'groups' here are only for the purpose of simplifying the schema structure for the reviewer and not part of the schema itself.
2. The 'FailureMode' has a schema of its own defined as further below.
3. A worked example for skill 'buy_item' as a JSON file, is provided with the code files, for the purpose of mapping, and easier understanding of the schema.

Groups (for explainability)	Field	Type	Description
Identity	skill_id	string	Unique identifier e.g. buy_item
	name	string	Human-readable name. 'Buy Item'
	description	string	1-2 sentence natural language descriptions. Used for embedding-based retrieval
	category	string	Primary category of the skill. For example: one of 'navigation', 'inventory', 'economy', 'combat, and 'crafting'
	version	semver	E.g. 1.0.0. Incremented on any changes to behaviour or schema fields
	status	enum	One of: candidate, active, deprecated
Invocation	preconditions	list[object]	State conditions that must hold before the skill can be invoked
Inputs	inputs.observations	list[object]	What the agent reads

			from the environment at each step
	inputs.memory	list[object]	What the agent reads from its short-term context window
	inputs.goal_params	dict	Task-specific parameters e.g. item_name: str
Outputs	outputs.state_changes	list[object]	Expected changes to game state on success
	outputs.artifacts	list[object]	Logs, flags, and signals emitted regardless of outcome
Termination	termination.success	list[object]	Conditions that signals the skill completed successfully
Execution Bounds	max_steps	int	Hard cap on actions before forced timeout. Prevents infinite loops
	timeout_ms	int	Hard timeout - skill is aborted if exceeded
Failure Handling	failure_modes	list[FailureMode]	Enumerated failure cases, each with a mode, severity, and recovery action
Metadata	games_validated	list[string]	Which game environment this skill has been tested and verified in
	success_rate	float	Rolling success rate across all live executions. Used for rot detection
	last_validated	date	Date of most recent validation run. Triggers re-validation

			if stale
--	--	--	----------

Field	Description
mode	Short name for failure type. E.g. 'item_not_found'
severity	P1 (game-breaking), P2(major), P3(minor), P4 (cosmetic)
recovery	Specific action to take. E.g. 'escape to known state and retry'

Q. Why does this representation enable re-use?

1. Use of Game-agnostic language
 - a. The description uses synonyms and related concepts so its embedding matches any natural language variation of the task across games.
 - b. Preconditions and termination conditions are falsifiable state assertions, checkable by a lightweight frame or event classifier, with no game-specific logic inside the skill itself.
2. Goal parameters decouple task variation from skill logic: The same skill handles 'buy health potion', 'buy sword', and 'buy shield' by varying a single parameter 'inputs.goal_params'. Without this, the library would need a separate skill per item per game.
3. The skill defines what; the executor defines how: The schema encodes intent and outcome, never interaction mechanism. Transferring to a new game only requires updating the executor, the skill definition is untouched. Please note that the executor is defined in Task 4.
4. Versioned: When a skill degrades in a new game, it is forked into a new version rather than overwritten, preserving a rollback point.
5. Category enables selective re-use: Skills are grouped by functional category, meaning transfer to a new game can also be scoped by category - only relevant categories need to be validated and transferred.

Task 2: Skill Discovery and Storage

Q1. How do you propose a candidate skill?

Candidate skills enter the library through two sources, ordered by increasing dynamism:

1. LLM decomposition (semi-dynamic): Given available game context, an LLM proposes candidate skills via in-context prompting in two steps.
 - a. First, the LLM is given the game context and asked to propose relevant functional categories. Each proposed category is compared against existing categories in the library - if a semantically similar category already exists it is skipped, otherwise it is added. This allows the category taxonomy to grow organically as new games are introduced.

- b. Second, given the updated category list, game context, and (optionally) task description, the LLM is prompted to generate candidate skills for each category conforming to the schema. Before a candidate is added, its description embedding is compared against all existing skill embeddings - if similarity exceeds a threshold (e.g. 0.9), the candidate is treated as a duplicate and discarded.
 - c. Note, in future once the category taxonomy is mature, Step 1 can be skipped and existing categories provided directly as context reducing overhead.
2. Gameplay-grounded proposal (fully-dynamic): During live gameplay, when the retrieval module finds no matching skill, the agent falls back to the baseline reactive policy. If the reactive policy successfully completes a sub-sequence, a background LLM process observes the action history and environment state and proposes it as a candidate skill. This captures interactions that offline decomposition would not anticipate. Note, the background LLM process is triggered once per completed reactive policy sub-sequence, not on every step, keeping overhead proportional to the rate of novel interactions rather than the total number of actions taken.
3. [Optional] Human seeding (static, one-time): The provided demonstration trajectories, sequence of (frame, action, event), completing tasks like 'Buy a potion' and 'Equip an item', are manually segmented into reusable sub-sequences and formalised as template skills to seed the library. These templates also serve as few-shot examples for all subsequent LLM-based proposals. In the future, domain experts could author the templates.

Q2. How do verify the skill is reliable?

A 'candidate' skill must pass a validation pipeline before being promoted to 'active'.

1. Run the candidate skill N times (say N=5) under varied conditions, different UI states, goal parameter values, preconditions and inputs, to test generalisation. Note, generating varied conditions requires either a configurable test environment or manually authored test cases, manual authoring is the MVP approach.
2. Each run is checked for hard failures: unhandled exceptions, infinite loops, or actions that leave the game in an unrecoverable state. Any candidate that produces execution errors is rejected. Note, hard errors may also indicate a bug in the game itself rather than a flaw in the skill, such cases are logged and flagged separately.
3. Each run checks whether the skill's termination conditions fired correctly: success condition reached within max_steps and timeout_ms. A candidate is promoted to 'active' only if it meets a minimum success rate threshold (e.g. > 80%) across all validation runs.

A candidate that passes all stages is promoted to 'active' and becomes retrievable. Validation results, success rate, failure reasons, and execution traces, are stored in the execution log against the skill and serve as reference for future re-validation runs.

Q3. How do you store and version skills?

Storage: All skills are stored in a single JSON file (skills.json), where each skill is defined as an object within the skills array and uniquely identified by its skill_id. A SQLite index maintains a queryable record of all skills including their status, version, and pre-computed description

embedding. JSON + SQLite is the deliberate MVP choice, simple, human-readable, and maintainable by a small team. A vector database such as ChromaDB or Pinecone is a natural future extension as the library scales beyond a few hundred skills.

Versioning: Version numbers follow MAJOR.MINOR.PATCH:

1. PATCH: minor fix that does not change behaviour, e.g. correcting a typo in a failure mode description.
2. MINOR: behavioural change that is backwards compatible e.g. adding a new failure mode or adjusting max_steps.
3. MAJOR: breaking change e.g. modifying preconditions or termination conditions in a way that affects when and how the skill is invoked.

Note: Version increments are manual, authored by whoever updates the skill definition. Also, in the future when a skill is updated, the previous version can be archived rather than overwritten. This means a skill that degrades in a new game can be forked into a new version while the previous version remains available as a rollback point.

Skill lifecycle: A skill moves through three states: candidate → active → deprecated
A skill is marked 'deprecated' when its live success rate drops below a threshold (e.g. 75%) or when it has not been validated within a defined window (e.g. 60 days). Deprecated skills are excluded from retrieval but retained for reference and potential revival.

Q4. How do you prevent library rot?

Now, to prevent library rot two mechanisms are utilised: detection mechanisms that catch degradation after it occurs, and prevention mechanisms that actively promote broad skill coverage before rot sets in.

Detection Mechanisms:

1. Success-rate monitoring: Every execution updates the skill's rolling 'success_rate' field. If it falls below threshold (e.g. 75%), the skill is used as a multiplier on its semantic similarity score inside 'rank_skills()' - naturally pushing it down in retrieval.
2. Time-to-complete monitoring: An exponential moving average of completion time is maintained per skill across execution logs. A performance anomaly is flagged when the EMA exceeds a tunable deviation threshold.
3. Regression: On every library write - any addition or major version update - validation re-runs for 'active' skills whose description embedding is semantically close to the new skill. The intuition is that a new skill only risks interfering with semantically similar skills (e.g. a new 'buy_item' variant is unlikely to break 'flee_combat'). Further, a full library validation runs on a fixed periodic schedule independently of writes.
4. [Optionally] Human review queue: Skills flagged by any of the above can be added to a review queue.

Prevention Mechanisms:

1. Stochastic retrieval: As detailed in Task 3, retrieval is intentionally non-deterministic. The LLM re-ranking call uses a high temperature (e.g. 0.7) instead of greedy - occasionally selecting lower-ranked but plausible candidates.
2. Entropy-based balancing: For each category c with n skills (for e.g. lets consider the category as 'fighting', then possible skills could be 'punching', 'kicking', and 'weapon load'), the system maintains a usage count k_i per skill and computes a normalised entropy:

$$H_{norm}(c) = \frac{-\sum_{i=1}^n p_i \log p_i}{\log n}, \quad p_i = \frac{k_i}{\sum_{j=1}^n k_j} \quad (1)$$

$H_{norm} = 1$ means all skills in the category are used equally. When H_{norm} drops below a threshold for a category, underused skills receive a score boost in the embedding pre-filter:

$$adjustedscore_i = score_i \times (1 + \alpha(1 - p_i)) \quad (2)$$

Where α is the boost strength, tunable scalar that controls how aggressively underused skills are promoted in the pre-filter score. Skills with low p_i receive a higher multiplier, nudging retrieval towards underused skills without overriding the LLM re-ranking stage. Usage counts are stored in a 'skill_usage' table and incremented on every retrieval call - tracking coverage, not just execution success.

Examples

Please note that the composite and goal-directed skills reference sub-skills by skill_id. Referenced sub-skills must themselves exist as active entries in the library. In a full implementation, the schema would include a sub_skills: list[skill_id] field to make this dependency explicit, deferred as future work.

Level	Skill	Game A	Game B	Reasoning
Atomic	open_map	Press M, map opens, fullscreen	Click minimap icon in corner, map slides in from left	Precondition: 'player is in navigable state' → Success Termination: 'map UI is visible' Note: neither condition encodes how the map is opened or where it appears
Composite	equip_item	Open inventory	Open character	Composed of

		with I → drag item to equipment slot	screen with C → click item → click confirm button	'open_inventory' and 'select_item' atomic skills. Goal_param 'item_name' is game-agnostic. Success Termination: 'item appears in equipment slot' holds in both games regardless of interaction mechanism
Goal-directed	complete_purchase_quest	Navigate to quest marker on map → buy item from shop → return to NPC	Open journal → accept question → buy item from merchant → talk to NPC	Composed of 'navigate_to_quest', 'buy_item', and 'interact_with_npc'. The composition and sequencing logic is identical across games. Only the executor changes

Task 3: Skill Retrieval

Pure rule or embedding based mechanisms are fast but have limitations - rules require explicit conditions and are limited in scope, embedding similarity is sensitive to keyword overlap rather than a contextual fit. Consequently, a hybrid retrieval pipeline is required:

1. Embedding pre-filter: The task description is first embedded and compared against all category embeddings - the top-c most relevant categories are selected. Please note that the category embedding is generated from a concatenation of the category name and all skill names within that category, ensuring the embedding captures the full semantic scope of the category rather than being dependent on the category label alone. Skill comparison is then restricted to active skills within those categories only. Note,

Without category pre-filtering: For a library of S active skills, the embedding pre-filter computes similarity between the task embedding and all S skill embeddings:

$$O(S \cdot d)$$

Where d is the embedding dimension.

With category pre-filtering:

First, compute similarity between the task embedding and all C category embeddings:

$$O(C \cdot d)$$

Then, select top- c categories and compute similarities only within those categories. So,

$$O(d \cdot \sum_{i=1}^c S_{c_i})$$

Now, the total cost becomes:

$$O((\sum_{i=1}^c S_{c_i} + C) \cdot d)$$

Now, if (and it is reasonable to say) $C + \sum_{i=1}^c S_{c_i} < S$, then the reduction is non-trivial.

Now, within the filtered set, each skill's similarity score is multiplied by its rolling 'success_rate' - naturally pushing degraded skills down - and underused skills receive an additional boost proportional to $(1 - p_i)$, as defined in equation 2, to promote entropy across skill space. The top- k candidates are passed to the LLM.

Note: In the future, a natural extension is a weighted embedding match incorporating the last N frame/event embeddings and last N skill embeddings alongside the task description - producing a context-aware pre-filter score. This is not implemented in the current prototype - the brief provides a frame as mock vectors, but applying similarity against pseudo embeddings would produce arbitrary boosts with no grounding. Once real embeddings are available, this extension is straightforward to plug in at the scoring step without any architectural changes.

2. LLM re-ranking: The task description, recent game event context (optionally, and as text), previously used skills in the current session (also optionally, and as text), and the top- k candidates are passed to an LLM with a structured prompt (optionally few-shot prompts). The LLM 'reasons' over which skill best fits the current situation - weighting factors that simple rule or embedding cannot capture. To prevent retrieval from converging on the same skills repeatedly, the LLM call uses a high temperature (e.g. 0.7). The LLM returns '(skill_id, score, rationale)' ordered by score descending.
3. Iterative Mechanism: The LLM has the option to reject all proposed candidates as unsuitable for the current task. In this case, the pipeline loops back to Stage 1 with an expanded search radius - incrementing both the number of categories to be considered ($c \rightarrow c + 1$ and the number of top- k ($k \rightarrow k + 1$) candidates proposed in the pre-filter Stage 1. This continues until either a suitable skill is found or a hard cap is reached (e.g. $c \rightarrow C$). At that point, if the LLM still rejects all candidates, the system falls back to the reactive policy as defined in Task 2.

Note: for the full implementation please refer to the attached code. Note that in the code, entropy-based score boosting is not applied as no usage history exists at initialisation. Also, all skills are assumed to be equally successful. The weighted embedding pre-filter is not implemented for reasons described above. Stochasticity is introduced via LLM temperature scaling.

Task 4: Skill Execution

Note that the agent executes a retrieved skill step-by-step. At each step, the agent observes the current game state, checks whether execution should continue, takes the next action, and verifies the outcome.

What is an Executor?

The executor is the component responsible for translating a retrieved skill's intent into concrete environment interactions. It operates as follows:

Inputs:

1. Retrieved skill (intent, goal parameters, preconditions, termination conditions)
2. Current frame from 'render()'
3. Latest event from the event stream e.g. {'type': 'ui', 'element': 'shop_button', 'state': 'clicked'}
4. Short-term memory queue

Outputs:

1. Selected action from 'action_space' passed to 'step(action)'
2. Termination signal - success, failure, or continue
3. Updated short-term memory entry

A snippet of pseudo-code is attached as follows. Also note the internal action space mechanism - how the executor maps skill intent to a concrete action from 'action_space' - depends on the game's action space specification which is not fully defined in the scope of this document.

```
#pseudo-code

state = env.reset()
while not terminated:
    frame = env.render()
    event = event_stream.latest()
    action = executor.select_action(skill, frame, event, memory)
    state = env.step(action)
    terminated, signal = executor.check_termination(skill, state, event, memory)
    memory.append(event)
```

Replanning triggers

These are checked at every step

1. Unexpected UI state: the screen shows something the skill did not anticipate.
2. Repeated action with no state change: the same action has been attempted N times with no observable effect, indicating the agent is stuck.
3. Timeout: elapsed time exceeds timeout_ms
4. Step limit: actions taken exceeds max_steps

5. Higher priority interrupt: an external event demands immediate attention e.g. the game enters a combat state mid-navigation, requiring the agent to switch to a combat skill before resuming the original task.

Recovery escalation.

When a trigger fires, recovery follows:

1. Retry in place: wait one step and retry the same action for pre-defined maximum retries.
2. Local recovery: executes the corrective action defined in the skill's failure mode entry for this specific trigger.
3. Return to known state: press Escape or equivalent until a recognised safe state is reached. Re-attempt the skill from the top.
4. Abort and replan: mark skill as failed, emit a detailed failure log, and return control to the higher-level planner to select a different approach. The failure log is written to the execution_log, capturing skill_id, failure reason, step count, and elapsed time.
5. Reflective replanning. Rather than blindly selecting the next skill, a structured reflection prompt is issued to the same LLM used for retrieval, reusing the same model. The prompt contains the current game state, recent action history from short-term memory, the failed skill and its failure reason, and the available skill library. The LLM diagnoses the failure and proposes one of three outcomes: a different skill from the library to attempt next, a modified goal parameter for the same skill, or a bug flag if repeated failures across recovery levels suggest an unreachable game state or missing trigger.

Short-term memory

The agent maintains a fixed-length FIFO (First In First Out) queue of the last N event strings from the environment's event stream, implemented as a 'deque(maxlen=N)'. On every step, the latest event is appended and the oldest is dropped. This queue is passed directly as 'context' argument to 'rank_skills()' at retrieval time and included verbatim in the LLM re-ranking prompt, where the model reasons over it alongside the candidate skills. The queue is cleared at session end.

Task 5: Evaluation

The evaluation is designed such that each metric maps to a specific failure.

Metric	What it measures	Failure mode it catches
Skill Success Rate Under UI Perturbations	% of executions reaching the success termination condition under UI perturbations - elements renamed, repositioned, or partially hidden	Skills that pass clean validation but break under slight UI variation - the most common real-world failure mode
Time to Complete	% of executions completing within 'timeout_ms'.	Infinite loops, server lag and resource leaks that do not cause hard failures.

Skill Transfer Rate	Skill success rate (% of executions reaching the success termination condition) measured in Game B for skills learned and validated in Game A - no schema change permitted.	Skills overfitted to Game A.
Regression Suite	On every library write, re-run the validation pipeline on the changed skill and all active skills whose description embedding is semantically close to it. Reports % of tested skills that still meet their success rate threshold. A full library validation runs on a fixed periodic schedule independently of writes	A skill update or addition silently degrading a semantically similar existing skill.
Retrieval Precision	For a benchmark set of tasks, whether the correct skills appear in the top-k results from 'rank_skills()'. Ground truth is established using an LLM-as-judge paradigm that evaluates skill-task avoiding the need for manual annotation.	Retrieval degrades as the library grows - more skills means more possibilities and harder ranking.

Relevant Literature:

1. [LLM Agents as Automated Game Testers](#)
2. [Leveraging LLM Agents for Automated Video Game Testing](#)
3. [Synergizing Code Coverage and Gameplay Intent: Coverage-Aware Game Playtesting with LLM-Guided Reinforcement Learning](#)
4. [GBQA: A GAME BENCHMARK FOR EVALUATING LLMS AS QUALITY ASSURANCE ENGINEERS](#)
5. [Potential Competitor](#)